GloVe 알고리즘 상세 분석

개요

이 문서는 GloVe(Global Vectors for Word Representation) 알고리즘의 구체적인 학습 과정을 단계별로 분석합니다. 이론적 배경에서 유도된 목적 함수가 실제로 어떻게 최적화되는지, 그리고 각 단계에서 어떤 수학적 연산이 수행되는지를 상세히 설명합니다.

용어 일관성: 이 문서에서는 01-glove-theory.md와 동일한 용어를 사용합니다. "동시출현 행렬", "임베딩", "벡터 표현" 등의 용어가 일관되게 적용되며, 수학 기호도 통일됩니다 (X_ij: 동시출현 빈도, f(X_ij): 가중치 함수, W/C: 임베딩 행렬).

알고리즘 전체 구조

입력과 출력

입력:

- 동시출현 행렬 X ∈ ℝ^(V×V) (V는 어휘 크기)
- 하이퍼파라미터: 임베딩 차원 n, 최대 반복 횟수, 학습률 등

출력:

- 단어 임베딩 행렬 G ∈ ℝ^(V×n)

알고리즘 의사코드

```
알고리즘: GloVe 학습

입력: 동시출현 행렬 X, 임베딩 차원 n, 최대 반복 횟수 max_iter

출력: 단어 임베딩 G
```

```
1. 초기화 단계:

a. 가중치 행렬 계산: weights = f(X)

b. 로그 동시출현 행렬 계산: log_X = log_of_array_ignoring_zeros(X)

c. 매개변수 초기화: W, C ∈ ℝ^(V×n), bw, bc ∈ ℝ^V



2. 반복 학습:

for iteration = 1 to max_iter:

a. 예측값 계산: pred = W @ C^T + bw + bc^T

b. 손실 및 그래디언트 계산:

- diff = pred - log_X

- weighted_diff = weights ⊙ diff

- 각 매개변수별 그래디언트 계산

c. AdaGrad 업데이트 적용

d. 수렴 조건 확인



3. 최종 임베딩 반환: G = (W + C) / 2
```

1단계: 초기화 과정

1.1 가중치 함수 계산

첫 번째 단계는 동시출현 행렬 X로부터 가중치 행렬을 계산하는 것입니다.

```
def _initialize(self, coincidence):
self.n_words = coincidence.shape[0]
bounded = np.minimum(coincidence, self.xmax) # xmax로 상한 제한
weights = (bounded / float(self.xmax)) ** self.alpha # 가중치 계산
log_coincidence = log_of_array_ignoring_zeros(coincidence)
return weights, log_coincidence
```

수학적 정의:

```
f(X_ij) = {
(min(X_ij, x_max) / x_max)^α if X_ij > 0
0 if X_ij = 0
}
```

매개변수:

- x_max = 100: 포화점 (일반적 값)
- α = 0.75: 지수 매개변수

목적:

- 매우 빈번한 동시출현의 영향력 제한
- 희소한 항목에 대해 0 가중치 부여
- 중간 빈도 항목들에 적절한 가중치 할당

1.2 로그 동시출현 행렬 계산

```
def log_of_array_ignoring_zeros(M):
log_M = M.copy()
mask = log_M > 0
log_M[mask] = np.log(log_M[mask]) # 0이 아닌 값만 로그 적용
return log_M
```

이 함수는 GloVe 이론에서 유도된 log(X_ij) 항을 안전하게 계산하는 역할을 합니다.

수학적 정의:

```
log_X_ij = {
log(X_ij) if X_ij > 0
0 if X_ij = 0
}
```

중요성:

- 목적 함수에서 log(X_ij) 항을 안전하게 계산
- 0 값에 대해서는 로그를 적용하지 않아 수치적 안정성 확보

1.3 매개변수 초기화

```
def _initialize_w_c_b(self, n_words, vocab, initial_embedding_dict):
self.W = randmatrix(n_words, self.n) # 단어 가중치 행렬
self.C = randmatrix(n_words, self.n) # 문맥 가중치 행렬
self.bw = randmatrix(n_words, 1) # 단어 편향
```

```
self.bc = randmatrix(n_words, 1) # 문맥 편향
```

Xavier Glorot 초기화:

```
def  randmatrix(m, n, random_seed=None):
"""Creates an m x n matrix of random values drawn using
the Xavier Glorot method."""
val = np.sqrt(6.0 / (m + n)) # Xavier Glorot 공식
np.random.seed(random_seed)
return np.random.uniform(-val, val, size=(m, n))
```

수학적 근거:

- 분산 = 2/(n_in + n_out): 그래디언트 소실/폭발 방지
- 균등 분포 사용: [-√(6/(m+n)), √(6/(m+n))]
- 각 층의 활성화 분산을 일정하게 유지

이 초기화 방법은 구현체 분석에서 실제 코드로 구현됩니다.

2단계: 예측값 계산

2.1 행렬 연산을 통한 예측

```
def  _make_prediction(self):
pred = np.dot(self.W, self.C.T) + self.bw + self.bc.T
return pred
```

수학적 표현:

```
pred_ij = W_i^T C_j + bw_i + bc_j
```

브로드캐스팅 활용:

- W @ C^T: (V×n) @ (n×V) → (V×V)
- bw: (V×1) → (V×V) (행 방향 브로드캐스팅)
- bc^T: (1×V) → (V×V) (열 방향 브로드캐스팅)

2.2 목적 함수와의 관계

예측값은 목적 함수의 핵심 구성 요소입니다:

```
J = Σ_{i,j} f(X_ij) × (pred_ij - log X_ij)²
```

여기서 pred_ij는 모델이 log X_ij를 얼마나 잘 예측하는지를 나타냅니다.

3단계: 그래디언트 계산

3.1 손실 함수 분해

```
def _get_gradients_and_error(self, predictions, log_coincidence, weights):
diffs = predictions - log_coincidence # 예측 오차
weighted_diffs = np.multiply(weights, diffs) # 가중치 적용
error = (0.5 * np.multiply(weights, diffs ** 2)).sum() # 총 손실
```

수학적 표현:

```
diff_ij = pred_ij - log X_ij

weighted_diff_ij = f(X_ij) × diff_ij

error = (1/2) × Σ_{i,j} f(X_ij) × diff_ij²
```

3.2 각 매개변수별 그래디언트

W에 대한 그래디언트

```
wgrad = weighted_diffs.dot(self.C)
```

수학적 유도:

```
∂J/∂W_i = Σ_j f(X_ij) × diff_ij × ∂pred_ij/∂W_i

= Σ_j f(X_ij) × diff_ij × C_j

= Σ_j weighted_diff_ij × C_j
```

행렬 형태: ∇_W J = weighted_diffs @ C

이러한 그래디언트 계산의 실제 구현은 [구현체 분석]에서 상세히 다룹니다.

C에 대한 그래디언트

```
cgrad = weighted_diffs.T.dot(self.W)
```

수학적 유도:

```
∂J/∂C_j = Σ_i f(X_ij) × diff_ij × ∂pred_ij/∂C_j
```

```
= Σ_i f(X_ij) × diff_ij × W_i

= Σ_i weighted_diff_ij × W_i
```

행렬 형태: ∇_C J = weighted_diffs^T @ W

편향에 대한 그래디언트

```
bwgrad = weighted_diffs.sum(axis=1).reshape(-1, 1) # 행별 합

bcgrad = weighted_diffs.sum(axis=0).reshape(-1, 1) # 열별 합
```

수학적 유도:

```
∂J/∂bw_i = Σ_j f(X_ij) × diff_ij × ∂pred_ij/∂bw_i

= Σ_j weighted_diff_ij


∂J/∂bc_j = Σ_i f(X_ij) × diff_ij × ∂pred_ij/∂bc_j

= Σ_i weighted_diff_ij
```

3.3 그래디언트 계산의 효율성

벡터화의 이점:

- 모든 그래디언트를 한 번의 행렬 연산으로 계산
- 반복문 없이 전체 어휘에 대해 동시 처리
- NumPy의 최적화된 BLAS 라이브러리 활용

계산 복잡도:

- W, C 그래디언트: $O(V^2 \times n)$

- 편향 그래디언트: $O(V^2)$
- 전체: $O(V^2 \times n)$

4단계: AdaGrad 최적화

4.1 AdaGrad 알고리즘 원리

AdaGrad는 각 매개변수별로 적응적 학습률을 사용하는 최적화 알고리즘입니다.

```
class AdaGradOptimizer:
def __init__(self, learning_rate, initial_accumulator_value=0.1):
self.learning_rate = learning_rate
self.initial_accumulator_value = initial_accumulator_value
self._momentum = None

def get_step(self, grad):
if self._momentum is None:
self._momentum = self.initial_accumulator_value * np.ones_like(grad)
self._momentum += grad ** 2  # 그래디언트 제곱 누적
return self.learning_rate * grad / np.sqrt(self._momentum)
```

4.2 AdaGrad 수학적 정의

누적 그래디언트 제곱:

```
G_t = G_{t-1} + g_t ⊙ g_t
```

적응적 학습률:

```
θ_{t+1} = θ_t - (η / √(G_t + ϵ)) ⊙ g_t
```

여기서:

- G_t: 시점 t까지의 그래디언트 제곱 누적
- η: 기본 학습률
- ε: 수치적 안정성을 위한 작은 상수 (initial_accumulator_value)
- ⊙: 원소별 곱셈

4.3 AdaGrad의 장점

1. **자동 학습률 조정**: 자주 업데이트되는 매개변수는 학습률 감소
2. **희소성 처리**: 드물게 업데이트되는 매개변수는 큰 학습률 유지
3. **수렴 안정성**: 그래디언트 크기에 따른 적응적 조정

4.4 매개변수 업데이트 적용

```
def _apply_updates(self, gradients):
if not hasattr(self, 'optimizers'):
self.optimizers = {
obj: AdaGradOptimizer(self.learning_rate)
for obj in ['W', 'C', 'bw', 'bc']
}
self.W -= self.optimizers['W'].get_step(gradients['W'])
```

```
self.C -= self.optimizers['C'].get_step(gradients['C'])

self.bw -= self.optimizers['bw'].get_step(gradients['bw'])

self.bc -= self.optimizers['bc'].get_step(gradients['bc'])
```

특징:

- 각 매개변수별로 독립적인 AdaGrad 인스턴스 사용
- 매개변수 형태에 관계없이 동일한 최적화 로직 적용

5단계: 수렴 조건과 조기 종료

5.1 손실 함수 모니터링

```
for iteration in  range(self.max_iter):

# ... 예측, 그래디언트 계산, 업데이트 ...

self.errors.append(error)

if error < self.tol:

break  # 허용 오차 이하로 감소시 조기 종료
```

5.2 수렴 기준

절대 허용 오차:

```
error < tol (기본값: 1e-4)
```

상대 허용 오차 (선택적):

```
|error_t - error_{t-1}| / error_{t-1} < relative_tol
```

5.3 학습 곡선 분석

일반적인 GloVe 학습 곡선의 특성:

1. **초기 급속 감소**: 첫 수십 반복에서 큰 폭 감소
2. **점진적 수렴**: 이후 천천히 최적값에 접근
3. **플래토 현상**: 최적값 근처에서 진동

6단계: 최종 임베딩 생성

6.1 단어와 문맥 벡터 결합

```
return  self.W + self.C
```

수학적 근거:

GloVe 논문에서는 단어 벡터 W와 문맥 벡터 C의 역할이 본질적으로 대칭적이라고 설명합니다. 따라서 최종 임베딩은 두 벡터의 합으로 계산됩니다.

6.2 대안적 결합 방법

실제 구현에서는 다음과 같은 방법들도 사용됩니다:

1. **평균**: (W + C) / 2
2. **단어 벡터만**: W
3. **문맥 벡터만**: C
4. **연결**: [W; C] (차원이 2배가 됨)

6.3 최종 임베딩의 특성

결합된 임베딩 G = W + C는 다음 특성을 만족합니다:

```
G_i^T G_j ≈ log X_ij + constant
```

이는 벡터 내적이 로그 동시출현 빈도와 비례함을 의미합니다.

알고리즘 복잡도 분석

시간 복잡도

반복당 계산량:

- 예측 계산: $O(V^2 + V \times n)$
- 그래디언트 계산: $O(V^2 + V^2 \times n)$
- 매개변수 업데이트: $O(V \times n)$

전체 복잡도: $O(\text{max_iter} \times V^2 \times n)$

공간 복잡도

필요한 메모리:

- 동시출현 행렬: $O(V^2)$
- 가중치 행렬: $O(V^2)$
- 매개변수: $O(V \times n)$
- AdaGrad 누적값: $O(V \times n)$

전체 공간 복잡도: $O(V^2 + V \times n)$

확장성 한계

어휘 크기 제한:

- V^2 메모리 요구사항으로 인해 대용량 어휘 처리 제한
- 일반적으로 V < 50,000 정도가 실용적 한계
- 희소 행렬 표현으로 어느 정도 완화 가능

수치적 안정성 고려사항

1. 로그 계산 안전성

```
def  log_of_array_ignoring_zeros(M):
log_M = M.copy()
mask = log_M > 0
log_M[mask] = np.log(log_M[mask]) # 0 값은 그대로 유지
return log_M
```

2. 그래디언트 폭발 방지

AdaGrad의 적응적 학습률이 그래디언트 폭발을 자동으로 방지:

```
step = learning_rate * grad / sqrt(accumulated_grad_squared + epsilon)
```

3. 언더플로우 방지

초기 누적값(initial_accumulator_value = 0.1)으로 분모가 0이 되는 것을 방지

실제 구현에서의 최적화

1. 벡터화 연산

모든 계산을 벡터화하여 반복문 제거:

```
# 비효율적: 이중 반복문
for i in range(V):
for j in range(V):
pred[i,j] = np.dot(W[i], C[j]) + bw[i] + bc[j]


# 효율적: 행렬 연산
pred = W @ C.T + bw + bc.T
```

2. 메모리 접근 패턴 최적화

NumPy의 브로드캐스팅과 연속 메모리 접근을 활용하여 캐시 효율성 향상

3. 희소성 활용

실제로는 동시출현 행렬의 대부분이 0이므로, 0이 아닌 원소에 대해서만 계산을 수행하는 최적화가 가능합니다.

결론

GloVe 알고리즘은 명확한 수학적 근거를 바탕으로 한 체계적인 최적화 과정을 통해 단어 임베딩을 학습합니다. 각 단계는 다음과 같은 핵심 역할을 수행합니다:

1. **초기화**: 안정적인 학습을 위한 매개변수 설정
2. **예측**: 현재 매개변수로 로그 동시출현 빈도 예측
3. **그래디언트**: 예측 오차를 바탕으로 개선 방향 계산

4. **업데이트**: AdaGrad를 통한 적응적 매개변수 조정

5. **수렴**: 최적값 도달 여부 판단

6. **결합**: 최종 임베딩 생성

이러한 체계적인 접근법을 통해 GloVe는 전역 동시출현 통계를 효과적으로 단어 벡터 공간에 인코딩할 수 있으며, 다양한 자연어 처리 태스크에서 우수한 성능을 보여줍니다. 이 알고리즘의 실제 구현은 구현체 분석에서, 성능 최적화는 성능 분석에서 다룹니다.

관련 문서

GloVe 알고리즘의 전체적인 이해를 위해 다음 문서들을 참조하세요:

- **GloVe 이론적 배경**: 이 알고리즘의 수학적 기초와 동기
- **Mittens 확장 개념**: 이 알고리즘을 확장한 도메인 적응 방법
- **구현체 코드 분석**: 이 알고리즘의 실제 구현 세부사항
- **성능 최적화 분석**: 알고리즘 구현의 성능 최적화 기법

학습 경로: 이론(01) → **알고리즘(02)** → 확장(03) → 구현(04) → 성능(05)